Lecture 23 → Application Layer: Design Patterns, DNS, HTTP, and QUIC

Opening question : How many application-layer protocols is your laptop speaking right now?

Axes of application protocol design

Axis	Option A	Option B
Interaction	request/response	push / pub-sub
State	stateless	stateful
Encoding	text (ASCII)	binary
Direction	pull	push
Connection	short-lived	persistent
Architecture	client–server	peer-to-peer

Examples by pattern

Protocol	Interaction	State	Encoding	Connection
HTTP/1.1	request/resp	stateless	text	semi-persistent
HTTP/2	request/resp	stateless	binary	persistent
DNS	request/resp	stateless	binary	short-lived (UDP)
SSH	stream	stateful	binary	long-lived
SMTP	command/resp	session	text	short-lived
WebSocket	duplex	stateful	framed	long-lived
BitTorrent	P2P	swarm	binary	many short

Stateless: The HTTP Cookie → each request stands alone, the server does not remember you between requests

2 families

Client-server → asymmetric roles

- Server listens, waits, Client initiates
- HTTP, DNS, SSH, SMTP, IMAP
- Scale: replication (CDNs), anycast, load balancers

Peer-to-peer → symmetric roles

- Every node is both client and server
- No central coordinator, or only a lightweight one
- Canonical example: BitTorrent

BitTorrent: mechanism design in a protocol

Problem: distribute a big file to many recipients without paying for all the bandwidth from one server

Design beats:

1. Chunks with hashes. File split into fixed-size chunks; each chunk has a cryptographic hash in the .torrent metadata. You can verify any chunk from any peer — no need to trust peers
2. Peer discovery. Trackers (hybrid) or DHT (fully decentralized)
3. Rarest-first chunk selection. Keep the rare chunks alive in the swarm
4. Tit-for-tat choking. Prefer peers who upload back to you; throttle free-riders

Popularity increases capacity. That is the P2P win

Why P2P doesnt win everywhere

P2P is a great fit for:

- Bulk file distribution (Linux ISOs, game patches, datasets)
- Censorship-resistant storage (IPFS)
- Consensus protocols (blockchain)

P2P is a bad fit for:

- Anything needing a single source of truth (account balance, authoritative DNS, email delivery)
- Real-time interactive services (chat, video call)
- Billing (who is the customer?)

DNS: the namespace

```
www.eng.bu.edu.
 |   |   |  |
 |   |   |  +-- TLD: edu
 |   |   +----- second level: bu
 |   +--------- subdomain: eng
 +------------- host: www
```

- Root: 13 server identities, hundreds of physical servers via anycast
- TLDs: edu, com, org, country-code (.uk, .jp), new gTLDs
- Authoritative: operated by the domain owner (or their DNS provider: Cloudflare, AWS Route 53)

DNS resolution chain

1. Stub resolver on laptop → recursive resolver (1.1.1.1 / 8.8.8.8 / your ISP)
2. Resolver cache hit? Done
3. Otherwise walk: root → “who handles edu?” edu → “who handles bu.edu?” bu.edu → “who handles eng.bu.edu?” eng.bu.edu → “what is www.eng.bu.edu?”
4. Answer bubbles back to the laptop

Recursive at the resolver. Iterative at authoritative servers. Most laptops never speak directly to a root or TLD server.

DNS record types

Type	Purpose
`A`	IPv4 address
`AAAA`	IPv6 address (four A's — four times as long)
`CNAME`	Alias to another name
`MX`	Mail exchange, with priority
`NS`	Name servers for a zone
`TXT`	Free-form text; SPF, DKIM, domain verification
`SOA`	Start of authority; zone metadata, TTLs

Live: dig +trace

```
dig www.eng.bu.edu +short              # just the IP
dig www.eng.bu.edu                     # full output
dig bu.edu MX +short                   # mail exchanges
dig bu.edu NS +short                   # name servers
dig +trace www.eng.bu.edu              # walk root -> TLD -> auth
dig -x 8.8.8.8 +short                  # reverse lookup (PTR)
```

DNS transport

Transport	Port	Use
UDP	53	Default. Small queries, small replies.
TCP	53	Large responses, zone transfers, DNSSEC.
TLS	853	**DoT** — DNS over TLS. Privacy.
HTTPS	443	**DoH** — DNS over HTTPS. Privacy, blends with web traffic.

What is 'the web'? → HTTP fetches one resource. One URL, one response

MIME types: what are these bytes?

The `Content-Type` header carries a **MIME type**:

MIME type	Used for
`text/html`	HTML documents
`text/css`	Stylesheets
`application/javascript`	JavaScript
`application/json`	JSON (APIs)
`image/png`, `image/jpeg`, `image/webp`	Images
`application/pdf`	PDF documents
`video/mp4`	Video
`application/octet-stream`	Unknown binary ("download me")

HTTP/1.1 → the baseline

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: curl/8.4.0
Accept: */*
```

Response:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
Connection: keep-alive

<!DOCTYPE html>...
```

HTTP/1.1 → key features

- Methods: GET, POST, PUT, DELETE, HEAD, OPTIONS, PATCH
- Status codes: 1xx info, 2xx ok, 3xx redirect, 4xx client error, 5xx server error
- Headers: key-value, extensible, case-insensitive
- Body framing: Content-Length or Transfer-Encoding: chunked
- Stateless — cookies provide the session illusio
- Keep-alive: reuse one TCP connection across requests

HOL problem. Pipelining (multiple requests on one connection) required responses to come back in request order. One slow response ⇒ everything behind it stalls. Browsers gave up and opened 6+ parallel connections

Live: curl with HTTP/1.1

```
curl -v --http1.1 https://example.com

> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.4.0
>
< HTTP/1.1 200 OK
< Content-Type: text/html
< Content-Length: 1256
...
```

HTTP/2 → binary and multiplexed

- Binary framing. Every message is a stream of binary frames. No more telnet debugging
- Multiplexing on one TCP connection. Many streams in flight at once; frames interleave. App-level HOL blocking solved
- HPACK header compression. Don't re-send the same headers on every request
- Server push — proactively send resources. Mostly deprecated in practicex4

Live: curl with HTTP/2

```
curl -v --http2 -o /dev/null https://www.cloudflare.com

* using HTTP/2
* h2h3 [:method: GET]
* h2h3 [:path: /]
* h2h3 [:authority: www.cloudflare.com]
...
< HTTP/2 200
< content-type: text/html
...
```

HTTP/3 and QUIC → HTTP/3 runs over QUIC. QUIC runs over UDP

Why UDP?

- TCP cannot be meaningfully changed in every middlebox on the internet
- NATs, firewalls, load balancers make strong assumptions about TCP
- UDP is dumb enough to mostly be left alone
- Move the reliable transport logic into user space, built on UDP

What QUIC provides:

- Reliable, ordered delivery per stream (TCP HOL gone)
- Multiplexed independent streams
- Built-in TLS 1.3 in the handshake (not a layer on top)
- 0-RTT resumption on reconnect
- Connection migration across IP changes

Live: curl with HTTP/3

```
curl -v --http3 -o /dev/null https://www.cloudflare.com

* Connected to www.cloudflare.com:443 via HTTP/3
* using HTTP/3
...
< HTTP/3 200
```

HTTP timeline

Year	Version	Transport	Wire	Fixes
1996	HTTP/1.0	TCP	text	—
1999	HTTP/1.1	TCP	text	keep-alive, host header
2015	HTTP/2	TCP	binary	multiplexing, HPACK
2022	HTTP/3	QUIC (UDP)	binary	TCP HOL gone, TLS built-in

Email is 3 protocols

Protocol	Port(s)	Role
SMTP	25 / 587 / 465	send (client → server, server → server)
IMAP	143 / 993	read mailboxes on a server
POP3	110 / 995	download mailboxes (legacy)

The webmail complication

Gmail is not speaking SMTP or IMAP to your browser

- Your browser speaks HTTPS to a Google web app
- Google's servers speak SMTP and IMAP on your behalf internally → to Google's own mail storage, and to external mail servers (outlook.com, bu.edu) for delivery
- The user-facing protocol is JSON-over-HTTPS, not email

Consequence: most users never see SMTP or IMAP

- Every webmail provider ships its own UI over its own private HTTP API
- The mail protocols between servers still run — and still carry massive backward-compatibility baggage
- This is one big reason email evolves slowly

Websocket → the escape hatch

WebSocket (RFC 6455): start with an HTTP *upgrade*, then switch to a persistent duplex frame channel.

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: ...
```

After the 101 Switching Protocols response, the connection stops being HTTP. Both ends send messages at any time.

Full circle : back to lecture

Content	MIME type	L2 said
HTML text	`text/html; charset=UTF-8`	UTF-8: ASCII compat, 1–4 B/char
Stylesheet	`text/css`	gzip/Brotli in transit
JavaScript	`application/javascript`	gzip/Brotli in transit
Photograph	`image/jpeg`	6.2 MB → 100 KB (**60**×)
Video stream	`video/mp4`	1.49 Gb/s → 5 Mb/s (**300**×)

One HTTP response , every layer

Layer	What it contributes
L2 — information	H.264 encodes frame: $\sim$1.49 Gb/s $\rightarrow$ 5 Mb/s
L3 — physical	Bits on fiber / radio / copper
L4 — wireless	OFDM subcarriers on 802.11ax
L5–L6 — error coding	CRC, FEC protect each frame
L7 — MAC	CSMA/CA arbitrates the air
L8 — Ethernet	Frame with preamble, MAC, FCS
L9 — WiFi	802.11 frame, probe/auth/assoc done
L13–L17 — network	IP datagram, TTL, routing, NAT
L18–L20 — transport	TCP (or QUIC) segment, cwnd, RTT
L21–L22 — tools	`ss`, `tcpdump`, sockets, Scapy, Mininet
L23 — application	HTTP/2 stream carrying `video/mp4`
$\rightarrow$ screen	One decoded 1080p frame. 33 ms later, the next.

<u>What tcpdump Could Read Today</u>

Every protocol in this lecture runs in cleartext on the wire by default:

- HTTP/1.1 headers, cookies, bodies — all visible
- DNS queries — visible to your ISP, your WiFi AP owner, governments
- SMTP between servers — often encrypted via STARTTLS, but originally plaintext

QUIC and DoH push encryption into defaults. The big answer is HTTPS = HTTP + TLS